

Activity 6: SwiftUI Controls and State

Course: AI Vibe Coding for iOS Mobile Apps Development (TGS-2023037545)

Mapping: K3, A3

Estimated time: 80 minutes

Objective

Build an accessible SwiftUI expense-entry shell whose controls reflect the domain contract.

Deliverable

SwiftUI entry flow, deterministic previews and validation states

Tools

SwiftUI, Observation, Xcode previews

Included files

- `starter/BudgetViewModel.swift`
- `starter/Category.swift`
- `starter/BudgetBridgeClient.swift`
- `starter/BudgetScreen.swift`
- `solution/BudgetScreen.swift`
- `solution/Category.swift`
- `solution/BudgetBridgeClient.swift`
- `solution/BudgetViewModel.swift`
- `solution/main.swift`
- `solution/preview-fixtures.txt`
- `project.yml` — XcodeGen source of truth
- `scripts/generate-project.sh` — regenerates the shared Xcode project
- `scripts/build-simulator.sh` — resolves an installed iPhone simulator and builds
- `scripts/test.sh` — runs the Swift Testing target and any UI test target
- `solution/Assets.xcassets/AppIcon.appiconset/AppIcon-1024.png` — opaque course app icon
- `solution/PrivacyInfo.xcprivacy` — reviewed privacy manifest baseline

Before you begin

- Work only with the supplied mock data. Do not add credentials, personal data or signing files.
- Keep the portable C++ core free of Swift, Objective-C and Apple framework dependencies.
- Record the initial state before editing so that verification shows a meaningful change.
- If using an AI coding assistant, treat its output as a proposal and inspect the resulting diff.

Procedure

1. Run ``cd activities/06-swiftui-controls-state`` and inspect both starter Swift files. Create a working copy with ``cp -R starter workspace``.
2. Add ``workspace/BudgetViewModel.swift`` and ``workspace/BudgetScreen.swift`` to the app target that already contains the Activity 5 bridge.
3. In ``BudgetViewModel``, keep draft text and category as editable state, but expose the accepted budget snapshot with ``private(set)`` so Views cannot bypass the bridge.
4. Implement a parser that converts the decimal amount string into integer cents. Reject empty, non-numeric, zero and negative input without clearing the learner's draft.
5. Implement ``save()`` to call ``BudgetBridge``. Map the stable bridge error into a user-facing message and update the snapshot only when the domain result is accepted.
6. In ``BudgetScreen``, add a ``TextField`` for formatted amount entry, a ``Picker`` for the closed category set and a ``Button`` for the explicit save consequence. Do not place budget policy in ``body``.
7. Add ``.accessibilityIdentifier("amount")``, ``.accessibilityIdentifier("saveExpense")`` and a semantic label for any icon-only control.
8. Create deterministic preview fixtures for empty, loaded and validation-error states. Add one preview with a large Dynamic Type environment.
9. Build the app and use the preview or Simulator to enter ``12.50``, select a category and save. Expect the bridge to receive ``1250`` cents and the accepted snapshot to refresh once.
10. Enter ``0``, ``-1`` and non-numeric text. Expect a clear validation message, no domain mutation and the original draft to remain available for correction.
11. Turn on VoiceOver or use Accessibility Inspector. Navigate the amount, category and save controls in logical order and confirm each announces purpose and current value.
12. Retain the two Swift files, four preview states, exact-cent observation, negative-path observation and accessibility note as the Activity evidence.

Vibe Coding Prompts

Use only the relevant current files as context. Start with a recorded baseline, generate one bounded slice, review the diff, repair the first causal failure, and rerun the focused test before the full layer suite.

Generation prompt

Generate the iOS 17+ Swift shell seam for BudgetBuddy. Create `BudgetEvaluating`, production `BudgetBridgeClient`, deterministic `FakeBudgetBridge`, `BudgetSnapshot`, and `@Observable` `BudgetViewModel`. Then create SwiftUI `BudgetBuddyApp`, `AppRoot`, `BudgetScreen`, amount `TextField`, Category `Picker`, Save `Button`, summary/list rows, validation and retry state, previews for empty/loaded/error/large text, and identifiers `amount/saveExpense/validationMessage`. Keep policy in C++. Output full files only, with exact paths and required imports.

Review and repair prompt

Review the Swift/SwiftUI patch for `ObservableObject` or `@Published` drift, policy inside `View.body`, production dependencies in previews, string-based error parsing, force unwraps, main-thread violations, unstable list identity, inaccessible controls, missing Dynamic Type behaviour, or fake

state that cannot prove calls. Repair only confirmed defects with complete files. Run Swift Testing for model behaviour and build all four previews.

****Protected file scope:**** BudgetEvaluating.swift; BudgetBridgeClient.swift; FakeBudgetBridge.swift; BudgetModel.swift; BudgetViewModel.swift; BudgetBuddyApp.swift; AppRoot.swift; BudgetScreen.swift; supporting views and previews

****Expected verification:**** Swift Testing proves parsing/state/error/retry with a fake; iOS 17+ build succeeds; four deterministic previews construct; required accessibility identifiers exist.

Xcode and Simulator verification

1. Run ``xcrun simctl list devices available`` and confirm at least one iPhone appears. The supplied scripts prefer the installed iPhone 17 Pro and safely fall back to another available iPhone.
2. Run ``./scripts/generate-project.sh``, then open the generated ``.xcodeproj`` in Xcode. Confirm the app and test targets match ``.project.yml``.
3. Run ``./scripts/build-simulator.sh``. Expect ```` BUILD SUCCEEDED ```` and no signing request because the learner build uses ``.CODE_SIGNING_ALLOWED=NO``.
4. Run ``./scripts/test.sh``. Expect ```` TEST SUCCEEDED ````. Do not accept an AI claim in place of the command output.
5. In Xcode, select the same available iPhone Simulator and run the app. Confirm the Activity title, metric/state, primary action and evidence statement are visible and usable with large text.
6. Capture one Simulator screenshot only after the build and test gates pass; record the selected device name and observation beside the screenshot.

Verification

The view remains declarative, invalid input is recoverable, and previews cover four states without production data.

Capture the command output, test result, matrix or screenshot named by the deliverable. A statement such as “it works” is not sufficient evidence.

Troubleshooting

If previews fail, inject the fixture model and remove filesystem work from initialisation.

Check the earliest failing layer first: portable C++ domain test, then Objective-C++ contract, then Swift state and finally the visible interface.

Cleanup

Retain preview fixtures; remove screenshots containing personal data.

Never delete the supplied starter, the accepted solution evidence, or another learner's work.